

Project 01: Federated SSO, SCIM Provisioning and API Integration

Project brief | Self-study edition | Duration: 7 days (about 20-25 hours)

Student	Darius Tamas
Format	Individual project, hands-on lab, delivered as a Git repository plus a short write-up
Prerequisites	Comfortable with Linux/WSL, Git, Python and basic SQL. No prior SSO experience needed.
Data policy	Fake data only. No real credentials, tokens or customer information anywhere in the repo.

1. Motivation

When a SaaS vendor onboards an enterprise customer, the customer's identity provider (Okta, Entra ID, Ping, Google Workspace) has to be connected to the vendor's application so that employees log in with company credentials and accounts are created, updated and removed automatically. Technical Account Managers own that connection end to end: they configure it, migrate customers onto it and debug it when it breaks. This project puts you on both sides of that connection so you understand exactly what travels over the wire and where it fails.

2. Learning objectives

By the end of the project you can:

- **LO1** - Explain and demonstrate an SP-initiated SAML 2.0 login, including every key element of the assertion.
- **LO2** - Implement an OpenID Connect (OAuth 2.0 authorization code + PKCE) login and compare it with SAML.
- **LO3** - Build and test a SCIM 2.0 /Users API and drive the full user lifecycle through it.
- **LO4** - Consume REST APIs robustly from Python: authentication, pagination, retries, idempotency, logging.
- **LO5** - Diagnose the most common SSO and provisioning failures from logs and captured traffic.
- **LO6** - Plan identity migrations (password to SSO, adding SCIM to an SSO tenant, switching identity provider).
- **LO7** - Describe how GraphQL differs from REST and query a GraphQL API.

3. Scenario

ShelfPilot is a fictional SaaS platform that helps retail teams share tasks and announcements with store staff.

Brightmart Retail, a fictional enterprise customer with headquarters and hundreds of stores, wants two things: employees signing in with their company identity (SSO), and accounts created and deactivated automatically when HR changes the directory (provisioning).

You play both roles. As the **customer's IT team** you run the identity provider (Keycloak). As the **ShelfPilot technical account manager** you build the service provider, connect the two, and then troubleshoot what goes wrong.

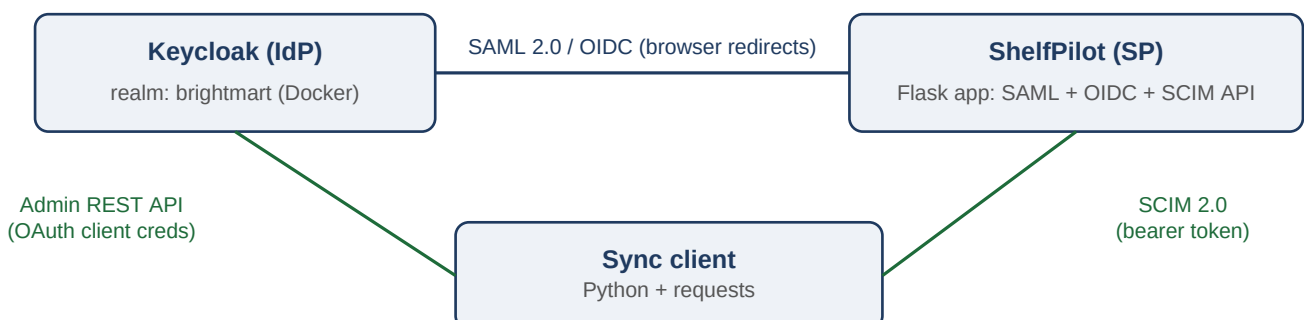


Figure 1. Target architecture. Everything runs locally in Docker and WSL.

4. Scope

In scope	Out of scope
• SAML 2.0 SP-initiated SSO • OIDC authorization code flow with PKCE • SCIM 2.0 Users endpoint (and Groups if time allows) • REST sync client against Keycloak Admin API and SCIM • Fault injection and incident write-ups • Migration runbook (paper exercise plus small demo) • GraphQL primer against a public API	• Production hardening, high availability, TLS termination • A polished UI (a plain page is fine) • Real customer data of any kind • Paid identity provider plans • Advanced SAML features such as encrypted assertions or artifact binding

5. Tools and environment

Area	Tool	Purpose
Runtime	WSL2 (Ubuntu), Docker and Docker Compose	Run Keycloak and supporting services
Identity provider	Keycloak (official container image)	Free, open-source IdP supporting SAML and OIDC
Application	Python 3.11+, Flask (or FastAPI)	ShelfPilot service provider and SCIM server
SAML / OIDC libs	python3-saml (or pysaml2), Authlib	SAML SP and OIDC client. python3-saml needs the xmlsec system libraries.
Data	SQLite (upgrade to PostgreSQL as a stretch goal)	User store; a good excuse to write real SQL
API testing	Postman or Bruno, curl, jq	Manual SCIM and REST calls, saved as a collection
Traffic inspection	SAML-tracer browser extension, browser dev tools, jwt.io (test tokens only)	Read assertions, redirects and tokens
Certificates and XML	openssl, xmllint	Generate and inspect signing certificates and metadata
Tunnel (optional)	ngrok or cloudflared	Expose your local SCIM API to a cloud IdP
Cloud IdP (optional)	Okta developer tenant or Microsoft Entra trial	Real-world provisioning test. Free tiers change, so check what is offered today.
Version control	Git and GitHub	Small commits, meaningful messages, .gitignore for secrets
Docs	Markdown, draw.io or Mermaid	README, diagrams, incident notes

Version note. Keycloak's admin bootstrap settings differ between versions (recent releases use `KC_BOOTSTRAP_ADMIN_USERNAME` and `KC_BOOTSTRAP_ADMIN_PASSWORD`, older ones `KEYCLOAK_ADMIN`). Check the docs for the version you pull, and pin the image tag in your compose file.

6. Work packages

WP0 - Environment and repository (Day 1)

Create a clean, reproducible base.

- Create the repo `shelfpilot-identity-lab` with folders `idp/`, `app/`, `sync/`, `docs/`, `postman/`.
- Write a `docker-compose.yml` that starts Keycloak; keep secrets in a git-ignored `.env` file.
- Create a Python virtual environment and a `requirements.txt`.

Done when: `docker compose up` brings Keycloak up on localhost and you can log in to its admin console.

WP1 - Customer identity provider (Day 1)

Model Brightmart Retail's directory.

- Create a realm brightmart with 6 fake users, two groups (hq-staff, store-managers) and custom attributes (department, store ID).
- Set a password policy and note how it would matter in a real migration.

Done when: users and groups exist and a test user can sign in to the Keycloak account console.

WP2 - SAML 2.0 single sign-on (Day 2)

Build the ShelfPilot service provider and connect it to Keycloak over SAML.

- Implement /saml/login, /saml/acs and /saml/metadata using python3-saml (or pysaml2).
- Register ShelfPilot in Keycloak as a SAML client: entity ID, ACS URL, name ID format, signing settings.
- Map attributes (email, first name, last name, groups) into the assertion.
- Capture a full login with SAML-tracer. Annotate the assertion: Issuer, Audience, Recipient, NameID, NotBefore/NotOnOrAfter, signature, attribute statements.

Done when: an SP-initiated login shows the user's attributes on a ShelfPilot page and you can explain every annotated element aloud without notes.

WP3 - OpenID Connect (Day 3)

Add a second login path using OIDC so you can compare protocols first-hand.

- Register a second client in Keycloak and add /oidc/login and /oidc/callback with Authlib, using the authorization code flow with PKCE.
- Decode the ID token (test tokens only). Identify issuer, audience, expiry, subject and claims.
- Write a one-page comparison of SAML and OIDC: flow, token format, typical use, common failure modes.

Done when: both login paths work and the comparison sheet is committed to docs/.

WP4 - SCIM 2.0 server (Day 4)

Give ShelfPilot the API a customer's IdP would call to provision users.

- Implement /scim/v2/Users: list with filter=username eq "...", startIndex and count; get by id; POST; PUT; PATCH; DELETE. Add /scim/v2/ServiceProviderConfig.
- Protect it with a bearer token. Return SCIM-format errors, including 409 with scimType: uniqueness for duplicate usernames.
- Store users in SQLite with an externalId and an active flag.
- Save a Postman/Bruno collection covering the whole lifecycle: create, read, update, deactivate (PATCH active=false), delete.

Done when: the collection runs green end to end and a deactivated user can no longer log in.

WP5 - REST sync client (Day 5)

Play the identity provider's role: read users from Keycloak and push changes to ShelfPilot.

- Get a token from Keycloak with the OAuth 2.0 client-credentials grant and call the Admin REST API to list users (handle pagination).
- For each user: look up by username, create if missing, update if changed, deactivate if disabled (idempotent, safe to re-run).
- Handle 401, 403, 404, 409, 429 (respect Retry-After) and 5xx with exponential backoff.
- Write structured logs (timestamp, user, action, result) and a summary line per run.

Done when: disabling a user in Keycloak and re-running the script deactivates them in ShelfPilot, and the log tells the whole story.

WP6 - JIT provisioning and migrations (Day 6)

Understand the alternatives to SCIM and the real projects TAMs run.

- **JIT:** create a ShelfPilot user on first SAML login from assertion attributes. Then delete the user in Keycloak and observe what JIT cannot do compared with SCIM.
- **Password to SSO:** seed 5 local password users, enable SSO and link accounts by email without creating duplicates. Write the rollback plan.
- **SAML to SCIM:** add provisioning to a tenant that already has SSO users. How do you match existing accounts (externalId vs userName)?
- **Switch IdP:** simulate moving from one identity provider to another with a second realm: new entity ID, new certificate, NameID consistency. Write a one-page cutover runbook.

Done when: the runbook exists and answers: what changes, what must not change, how to test, how to roll back.

WP7 - GraphQL primer (Day 7, 2-3 hours)

Close the gap on the one API style you have not used yet.

- Read the introductory guide on graphql.org (schema, queries, mutations, variables).
- Query a free public GraphQL API (for example a countries or characters API; endpoints change, so check one is still live). Write 3 queries with variables, one using nested fields.
- Note that GraphQL usually answers HTTP 200 even on errors, with an errors array. Write a half-page comparison with REST.

Done when: you can explain over-fetching, under-fetching and introspection in two minutes.

7. Fault-injection lab (Tier 3 practice)

Real escalations are rarely about building things; they are about finding what broke. For each scenario below, break your system on purpose, capture the evidence (logs, SAML-tracer, HTTP traces), then write a five-line incident note: **symptom, evidence, root cause, fix, prevention**. Complete at least 6 of the 9.

#	Break this	What you should observe and learn
1	Wrong ACS URL or entity ID in the IdP client	Login rejected at the IdP or assertion never accepted. Learn to compare SP metadata with IdP client settings.
2	Clock skew: shift the SP container clock by 10 minutes	Assertion rejected as not yet valid or expired. Learn NotBefore/NotOnOrAfter and time tolerance.
3	Rotate the IdP signing key without updating the SP	Signature validation fails. Learn why certificate rotation needs an overlap plan.
4	Rename or remove a mapped attribute (for example email)	Blank or wrong profile, or a failed login. Learn attribute mapping and how to verify it in the assertion.
5	Change NameID format (email to persistent ID)	Users appear duplicated or unlinked. Learn why NameID must stay stable across migrations.
6	POST a user whose userName already exists	409 with scimType uniqueness. Learn how IdPs interpret conflicts and how to match existing accounts.
7	Send PATCH deactivation in different shapes	Some IdPs vary operation capitalization or send booleans as strings. Make the server tolerant and confirm the user is really blocked.
8	Expired or wrong bearer token	401 vs 403: authentication vs authorization. Learn how to tell them apart in logs.
9	Hammer the API to trigger 429 (add a simple rate limiter)	Learn Retry-After, backoff and why naive retries make outages worse.

8. Schedule

Day	Focus	Output
1	Reading (SAML and OIDC basics), WP0, WP1	Repo, Keycloak running, realm and users
2	WP2 SAML SSO	Working SAML login, annotated assertion
3	WP3 OIDC, start WP6 (JIT)	OIDC login, SAML vs OIDC comparison sheet
4	WP4 SCIM server	Passing Postman collection
5	WP5 sync client, optional cloud IdP test	Sync script with logs, demo of deactivation
6	Fault injection, WP6 migrations	6+ incident notes, cutover runbook
7	WP7 GraphQL, README, diagram, write-up, mock interview	Finished repository and 1-page summary

Optional stretch: point a real Okta or Entra tenant at your SCIM endpoint through a tunnel and record what differs from your own client (request shapes, retries, error handling). Only do this if the earlier packages are solid, and only with fake users.

9. Deliverables

- Git repository with a README explaining how to run everything from scratch in under 10 minutes.
- `docker-compose.yml`, the ShelfPilot app, the SCIM server and the sync client.
- Postman/Bruno collection for the SCIM lifecycle.
- `docs/`: architecture diagram, annotated SAML assertion, SAML vs OIDC vs SCIM comparison, at least 6 incident notes, migration cutover runbook, REST vs GraphQL note.
- A one-page summary: what you built, what broke, what you would do differently.
- Hygiene: no secrets committed, fake data only, clear commit history.

10. Self-assessment rubric

Criterion	Meets expectations when...
SAML and OIDC	Both logins work and you can explain each token or assertion field and what breaks if it is wrong.
SCIM	Full lifecycle passes, errors are SCIM-compliant, deactivation actually blocks access.
REST quality	Client is idempotent, paginates, retries sensibly and logs enough to reconstruct a run.
Troubleshooting	Incident notes state a root cause backed by evidence, not just a fix.
Migrations	Runbook covers matching keys, testing, rollback and communication to the customer.
Communication	You can explain the whole system to a non-technical account manager in five minutes.

11. Reading list

- OASIS: *Security Assertion Markup Language (SAML) V2.0 Technical Overview*
- OpenID Connect Core 1.0 and RFC 6749 (OAuth 2.0), RFC 7636 (PKCE), RFC 6750 (bearer tokens)
- RFC 7642 (SCIM concepts), RFC 7643 (SCIM schema), RFC 7644 (SCIM protocol)
- Keycloak documentation: Server Administration Guide (SAML and OIDC clients)
- Okta developer documentation and Microsoft Learn on SCIM provisioning (real-world quirks)
- graphql.org: Introduction to GraphQL

12. Interview talking points

Prepare a 60-second answer for each, using your own lab as the example:

- Walk me through an SP-initiated SAML login, step by step.
- SCIM vs JIT provisioning: when would you use each, and what does JIT miss?
- After a certificate rotation, users cannot log in. What do you check first?
- How would you migrate a customer from password login to SSO without creating duplicate accounts?
- What is the difference between a 401 and a 403, and how do you tell which one you are seeing?
- REST vs GraphQL: what changes for a customer building an integration?

Write it up honestly. This is a self-directed lab project, and that is how it should appear on your CV: for example, *Self-directed lab: built a SAML/OIDC service provider and SCIM 2.0 server integrated with Keycloak; documented failure scenarios and a migration runbook*. Do not present it as coursework or as work for a customer. Only add a claim (for example SAML, SCIM, OAuth) after you have completed the matching work package, and be ready to demo it from your repository.